AIGym

System Description

Abstract

This document describes the AIGym support system of the Eclipse Arrowhead 5.2.2 framework, as implemented in `ah5-AIGym-java-spring` of the primary Java source tree (`Eclipse-Arrowhead-java`) and deployed with the standalone Compose definition of that folder.

Contents

1 Overview **3**
1.1 Significant Prior Art 4
1.2 How This System Is Meant to Be Used 4
1.3 System functionalities and properties 4
1.4 Important Delimitations 5

2 Services **6**
2.1 Produced services 6
2.2 Consumed services 6

3 Security **7**
3.1 Security model 7

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the AIGym system. AIGym is an experimentation system for machine learning in the local cloud: it collects training samples, predicts with k nearest neighbours and consumes the `sysMonitorData` stream.

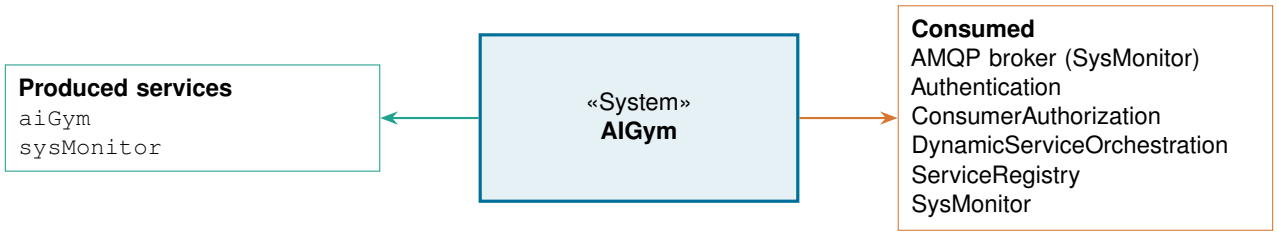


Figure 1: SysML-style block view of AIGym with its produced and consumed services.

The rest of this document is organized as follows. In Section 1.1, we reference major prior art capabilities of the system. In Section 1.2, we describe the intended usage of the system. In Section 1.3, we describe fundamental properties provided by the system. In Section 1.4, we describe delimitations of capabilities of the system. In Section 2, we describe the abstract service functions consumed or produced by the system. In Section 3, we describe the security capabilities of the system.

1.1 Significant Prior Art

New in this workspace.

1.2 How This System Is Meant to Be Used

Clients add samples and request predictions; AIGym orchestrates `sysMonitorData` and keeps the latest observations.

1.3 System functionalities and properties

1.3.1 Functional properties of the system

- k-NN model store.
- sysMonitorData subscription.
- sysMonitor publication with `trainingSample` and `inference` observations.

1.3.2 Configuration of system properties

The system is configured with Spring properties: the bundled `application.yml` reads environment variables, and the standalone deployment overrides it with `config/application.properties` (copied into the image, `spring.config.additional-location`), whose passwords come from Compose file secrets. The properties that shape its behaviour are:

Property (variable)	Default	Meaning
`server.port` (`PORT`)	8478	HTTPS port (mutual TLS, `client-auth=need`).
`arrowhead.advertised-host` (`ADVERTISED_HOST`)	aigym	Registered address.
`arrowhead.registry-url` (`SERVICE_REGISTRY_URL`)	`https://serviceregistry:8443`	The only configured Arrowhead endpoint; Authentication and the orchestrator are looked up there.
`arrowhead.authentication-url` (`AUTHENTICATION_URL`)	(empty)	Optional override of the Authentication lookup.
`arrowhead.orchestration-ttl-seconds` (`ORCHESTRATION_TTL_SECONDS`)	300	Reuse of orchestration results.
`arrowhead.mqtt-host` / `-port` (`MQTT_HOST` / `MQTT_PORT`)	mqtt-broker / 8883	sysMonitorData broker.
`arrowhead.amqp-host` / `-port` / `-exchange` (`AMQP_*`)	amqp-broker / 5671 / arrowhead	sysMonitor publication.
`arrowhead.status-interval-ms` (`STATUS_INTERVAL_MS`)	60000	Status publication interval.
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate, reloaded on change, fail closed (standalone deployment: `/crl/ca.crl`).

Secrets (keystore and identity passwords) must be supplied by the deployment; the bundled `application.yml` still has development fallbacks, which the standalone configuration overrides: it reads them from per-deployment Compose file secrets under `/run/secrets`.

1.3.3 Data stored by the system

Models and observations in memory.

1.3.4 Non functional properties

Small in-memory models; not for production learning.

1.3.5 Stateful or stateless

Volatile.

1.4 Important Delimitations

Experimental; no persistence.

2 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

2.1 Produced services

Service	Version	Interfaces	SD	IDD
`aiGym`	1.0.0	HTTP(S)	[1]	[2]
`sysMonitor`	1.0.0	AMQP(S)	[3]	[4]

The HTTP interface uses `generic_https` with `CERT_AUTH`; `GET /health` is not registered.

2.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	systemDiscovery, serviceDiscovery	register, lookup	Registration; core lookup.	always
Authentication	identity	identity-login, identity-verify	Own token; caller verification.	always
Consumer Authorization	authorization	verify	Per-call authorization.	always
DynamicService Orchestration	service Orchestration	pull	sysMonitorData.	always
SysMonitor	sysMonitorData	subscribe	Observation stream.	always
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status, trainingSample, inference	Own observations.	always

Only Authentication's `identity` and the orchestrator are looked up in the ServiceRegistry; other services are reached through DynamicServiceOrchestration (results cached), and every such call needs a Consumer-Authorization grant.

3 Security

The system can run in an insecure mode (`server.ssl.enabled=false`, plain HTTP, identity declared by the requester) for development, and in the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The standalone deployment of this folder (`config/application.properties` and `compose.yaml`) runs the secure mode.

3.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates; MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the standalone deployment. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** mutual TLS with a system or operator certificate of the cloud and an identity token of the same system, verified with Authentication; declared identities are not accepted. Client certificates are checked against the CRL.
- **Authorization of produced services:** per operation by caller lists in the implementation and, where stated, by ConsumerAuthorization.
- Mutual TLS, identity token bound to the certificate, ConsumerAuthorization per operation.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

4 References

[1] J. Delsing, “aiGym – Service Description (AIGym),” Eclipse Arrowhead, SD 5.2.2, 2026, aIGym-v5.2.2-aiGym-SD.pdf.

[2] ——, “aiGym HTTP/TLS/JSON – Interface Design Description (AIGym),” Eclipse Arrowhead, IDD 5.2.2, 2026, aIGym-v5.2.2-aiGym-HTTP-IDD.pdf.

[3] ——, “sysMonitor – Service Description (AIGym),” Eclipse Arrowhead, SD 5.2.2, 2026, aIGym-v5.2.2-sysMonitor-SD.pdf.

[4] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (AIGym),” Eclipse Arrowhead, IDD 5.2.2, 2026, aIGym-v5.2.2-sysMonitor-AMQP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	